

UNIVERSIDAD

Sistema de Informacion Institucional - SII

DOCUMENTACION TECNICA FINAL

Arquitectura, APIs y Estructura del SII

Version: v1.0

Fecha: 29 de mayo de 2026

Equipo de QA - Aseguramiento de Calidad

1. Descripcion General del Sistema

El SII (Sistema de Informacion Institucional) es una aplicacion web de tipo cliente-servidor desarrollada para el Universidad. Gestiona los procesos academicos y administrativos de la universidad mediante una API RESTful en el backend y una interfaz web estatica en el frontend.

2. Arquitectura del Sistema

2.1 Stack Tecnologico

Capa	Tecnologia	Version
Backend	Node.js + Express.js	Node 18+ / Express 4.22
Base de datos	PostgreSQL (Neon cloud)	pg v8.20
Frontend	HTML5, CSS3, JavaScript vanilla	ES2020+
Tipografia	Google Fonts - DM Sans	Variable
CORS	cors npm package	2.8.6

2.2 Estructura de Directorios

backend/

- server.js — Punto de entrada, configuracion de middlewares y rutas.
- db.js — Pool de conexion PostgreSQL.
- middleware/autorizar.js — Middleware de autorizacion por roles.
- routes/ — Endpoints REST por modulo (alumnos, docentes, grupos, materias, horarios, inscripciones, calificaciones, aulas, carreras, roles, usuarios, aspirantes).

frontend/

- css/ — Hojas de estilo (styles.css, gestion.css, portales.css).
- js/ — Logica de cada modulo (api.js, auth.js, dashboard.js, gestion-*.js).
- Paginas HTML por modulo y rol de usuario.

3. Modelo de Datos Principal

Tablas principales:

Tabla	Descripcion
usuarios	Cuentas de acceso con rol (admin, docente, alumno, aspirante).
alumnos	Padron de estudiantes con matricula unica (formato 22XXXXXX).
docentes	Personal docente con numero de empleado unico.
grupos	Grupos academicos por grado y turno.
materias	Catalogo de materias con clave, nombre y creditos.
materia_grupo	Pivot: asignacion de materia a grupo con docente y horario.
horarios	Slots de horario por materia_grupo: dia, hora_inicio, hora_fin.
inscripciones	Inscripcion de alumno a materia_grupo por periodo.
calificaciones	Calificaciones parciales (1-3) por inscripcion.
aspirantes	Datos del proceso de admision con estatus y documentos.
vista_calificaciones	Vista SQL que consolida datos de inscripciones y calificaciones.

4. API REST - Endpoints Principales

4.1 Autenticacion

Metodo	Endpoint	Descripcion	Roles
POST	/login	Autenticacion y generacion de token	Publico

4.2 Alumnos

Metodo	Endpoint	Descripcion	Roles
GET	/alumnos	Lista todos los alumnos activos	admin, docente
GET	/alumnos/boleta/:uid	Boleta de calificaciones por usuario	alumno, admin
GET	/alumnos/reticular/:uid	Avance reticular completo	alumno, admin
GET	/alumnos/horario/:uid	Horario semanal del alumno	alumno, admin
POST	/alumnos	Crear alumno (genera matricula)	admin
PUT	/alumnos/:id	Editar datos del alumno	admin
DELETE	/alumnos/:id	Desactivar alumno (soft delete)	admin

4.3 Grupos y Horarios

Metodo	Endpoint	Descripcion	Roles
GET	/grupos	Lista grupos activos	Publico
POST	/grupos/:id/materias	Asignar materia a grupo	admin
POST	/grupos/:id/materias/:mg/horarios	Agregar slot de horario	admin
DELETE	/grupos/:id/materias/:mg/horarios/:h	Eliminar slot de horario	admin

5. Mecanismo de Autorizacion

El SII implementa un sistema de autorizacion basado en tokens Base64. El middleware autorizar.js verifica el encabezado Authorization en cada peticion protegida.

Formato del token:

"id:usuario:rol" codificado en Base64.

Proceso de verificacion:

1. El cliente incluye el token en el header: Authorization: Bearer <token>.
2. El middleware decodifica el Base64.
3. Extrae id, usuario y rol.
4. Verifica que el rol tenga permiso para la ruta solicitada.
5. Adjunta req.usuario para uso en los handlers.

6. Funciones SQL Especiales

Funcion/Vista	Descripcion
generar_matricula()	Genera matricula unica con formato 22XXXXXX usando secuencia SQL.
generar_numero_empleado()	Genera numero de empleado con formato 11XXXXXX.
vista_calificaciones	Vista que consolida inscripciones, calificaciones, promedios y estados por alumno-materia-periodo.

7. Consideraciones de Seguridad

- Las contraseñas se almacenan en texto plano (pendiente implementacion de hashing).
- El token Base64 no es criptográficamente seguro; se recomienda migrar a JWT.
- La cadena de conexion a BD no debe estar en el codigo fuente (usar variables de entorno).
- El CORS esta habilitado para todos los origenes (*); restringir en produccion.

8. Despliegue Recomendado

6. Clonar el repositorio del proyecto.
7. Instalar dependencias: npm install en /backend.
8. Ejecutar el servidor: npm start (node server.js).
9. Servir el frontend con cualquier servidor estatico (nginx, Apache, Live Server).
10. Configurar variables de entorno para la cadena de BD en produccion.